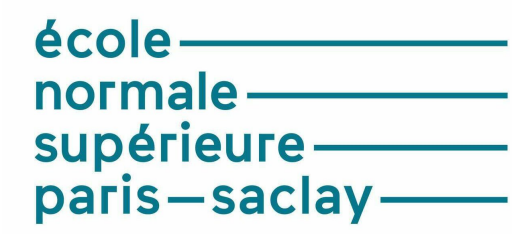


Bridging Backward and Forward Reasoning: MiniMindAgent — A Tree-Memory Framework for Small Language Model-Based AI Tool Agents

Yassine Kaddami

Internship at Yesop

ENS Paris Saclay

Company Supervisor: Mr. Raphael Salmon

School Supervisor: Pr. François Malgouyres

Internship Starting Date: 13/05/2025

Internship report MVA

CONTENTS

Bridging Backward and Forward Reasoning: MiniMindAgent — A Tree-Memory Framework for Small Language Model-Based AI Tool Agents*

Yassine Kaddami

12/09/2025

1 INTRODUCTION

The development of autonomous AI agents has been greatly accelerated by the rapid progress of language models, enabling systems that can reason, plan, and act through tool use. However, most successful approaches to date rely on large language models (LLMs), which are computationally expensive and difficult to scale. Small language models (SLMs) represent a promising alternative, but their limited reasoning capacity and memory handling often prevent them from performing well on complex multi-step tasks. Traditional paradigms such as ReAct Yao et al. (2023b) employ linear reasoning and memory structures, yet they struggle with decomposing goals and with extending reasoning and self-questioning processes to fully uncover the path toward a solution. This motivates the search for new reasoning and memory strategies that can unlock the potential of SLM-based agents.

I did my research internship at Yseop, an innovative company specializing in Natural Language Generation (NLG) software and AI-driven automation. Yseop develops applications that enhance efficiency and decision-making across industries, with a strong focus on the pharmaceutical sector. The internship took place within the context of a three-year

*Acknowledgement: I would like to express my gratitude to my supervisors, Mr. Raphael Salmon and Pr. François Malgouyres, for their constant support throughout my internship. I am also grateful to Mrs. Dominique Mariko, who supervised me during the first two months. Finally, I would like to warmly thank Mr. Maxime Guillaume for his valuable help and assistance.

research project named *Genysis* led by the Yseop Innovation Team, aiming to build an Automation Assistant for clinical, pre-clinical, and pharmacovigilance tasks. One of the key challenges in this project is the orchestration of agents using efficient small models, making reasoning and memory strategies for SLMs a central research problem.

In this context, this study addresses the following research question: How can the performance of a tool agent relying solely on small language models be improved for complex, multi-step tasks? To investigate this, I designed MiniMindAgent, a framework where the agent begins by reasoning backwards from the desired final tool call, decomposing its arguments into subtasks. If an argument cannot be directly obtained, the agent spawns a forward exploratory action whose result is either accepted as sufficient or used to generate further backward subtasks. All reasoning and actions are stored in a tree memory, where nodes are executed automatically once all inputs are satisfied, enabling simultaneous planning and execution. The framework is empirically evaluated on an environment based on the GTA benchmark Wang et al. (2024a), a suite of realistic tool-use challenges, with performance compared against ReAct Yao et al. (2023b) paradigm using linear memory as the baseline.

Our experimental results highlight both the challenges and opportunities of reasoning with small language models. Baseline comparisons between OnlyAct and ReAct confirms that incorporating "thinking" improves performance for smaller models (1.5 billion parameters). We also observed that linear forward reasoning reaches a bottleneck when addressing multi-step, complex tasks. However, our exploration of MiniMindAgent suggests that more sophisticated paradigms—combining forward and backward reasoning—can partially overcome these limitations and enable richer forms of problem solving. Although preliminary and constrained by limited resources, these findings indicate that lightweight models can serve as valuable testbeds for developing novel agent paradigms. Furthermore, the emerging insights point to several promising research directions, including new training methodologies. This work therefore aims not only to benchmark current limitations but also to outline a trajectory toward more robust and adaptive reasoning systems.

During this internship, my main contributions were as follows:

- Developed a taxonomy of tool-agent problems, allowing the research to focus on a well-defined and tractable subproblem.

- Conducted a comprehensive literature review covering small language model (SLM) agents, reasoning paradigms, and large language model (LLM) reasoning and training strategies.
- Designed and implemented an experimental environment for testing AI agents, built on open-source datasets and tailored to the selected research problem.
- Applied baseline paradigms and memory systems within this environment to establish reference points for comparison.
- Designed and implemented a novel agent paradigm, achieving promising results compared to the baseline frameworks.
- Delivered an innovation of strategic importance for Yseop, providing empirical evidence to guide decisions on the choice of AI agents orchestration frameworks for medical and pharmaceutical Natural Language Generation use cases.

2 BACKGROUND KNOWLEDGE

In this section, we briefly review the core technical concepts, primarily covered in MVA courses, that form the foundation of my internship project and are further extended in this work.

2.1 Related MVA courses

The internship research is tightly connected to the course "Large Language Models: introduction and application for code". Not only the key concepts studied in this course are primordial to understand all the research papers related to my topic and to guide my research plan such as Attention Mechanism, Policy Optimization, Power Laws, Tokenization, Positional Encoding, but this internship subject expands one of the topics encountered during the course, which is the the reasoning capabilities of Language Models. In particular, I explored a novel framework for developing a reasoning and planning system, where the model generates tool-use requests and produces answers for the user based on observations obtained from tool calls. It also opens new research area to use new policies to train an orchestrator model on that same framework.

2.2 Core concepts

Language Models A language model is a probabilistic model designed to predict the continuation of a sequence of text. Given some input text, the model outputs a distribution over possible next elements, thereby allowing it to generate coherent sequences word by word. For example, if the input is "The cat is on the", a well-trained language model will assign higher probability to continuations such as "mat" or "sofa" compared to unlikely completions such as "quantum".

The basic unit of input for a language model is a **token**. A token is not necessarily a full word; depending on the tokenization scheme, it can represent an entire word ("cat"), a subword piece ("ing"), or even a single character. Formally, the vocabulary $\mathcal{V}$ is the set of all tokens that the model can produce. Text sequences are mapped into token sequences

$$x_{1:n} = (x_1, x_2, \ldots, x_n), \quad x_t \in \mathcal{V}.$$

A language model assigns a probability to the entire sequence by predicting each token conditioned on all the tokens before it:

$$p_\theta(x_{1:n}) \;=\; \prod_{t=1}^{n} p_\theta(x_t \mid x_{1:t-1}),$$

where θ denotes the parameters of the model. At each step t, the model produces a probability distribution over the vocabulary, and the most likely or a sampled token is chosen as the next element in the sequence.

Training a language model consists of adjusting the parameters θ so that the model assigns high probability to real text sequences from the training corpus. This is typically achieved by minimizing the negative log-likelihood, also known as the cross-entropy loss:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{n_i} \log p_\theta\Big(x_t^{(i)} \mid x_{1:t-1}^{(i)}\Big),$$

where N is the number of training sequences and n_i the length of sequence i.

Embeddings. Neural networks cannot operate directly on discrete tokens, since each token is simply an index in the vocabulary $\mathcal{V}$. To make tokens usable by the model, they are mapped into continuous vectors called **embeddings**. Each token $x_t \in \mathcal{V}$ is associated

with a vector $e_t \in \mathbb{R}^d$, where d is the embedding dimension. The embedding layer can be viewed as a matrix

$$E \in \mathbb{R}^{|\mathcal{V}| \times d},$$

where each row corresponds to the vector representation of one token in the vocabulary. Given a token index x_t, the embedding operation selects the corresponding row of E:

$$e_t = E[x_t].$$

The resulting sequence of embeddings $(e_1, e_2, \ldots, e_n)$ forms the continuous input representation of the text, which captures semantic similarity (e.g., "dog" and "cat" embeddings are closer than "dog" and "car").

Positional Encoding. While embeddings represent the identity of tokens, they do not contain information about the *order* of the tokens in the sequence. However, word order is crucial for natural language. For example, "dog bites man" and "man bites dog" contain the same tokens but have very different meanings.

One way of addressing this is by adding **positional encodings** to embeddings, so that each input vector reflects both the token identity and its position. Let $p_t \in \mathbb{R}^d$ denote the positional encoding of position t. The final input to the Transformer is then

$$z_t = e_t + p_t.$$

Attention Mechanism The core idea of attention is to let each token in a sequence selectively focus on other tokens that are most relevant for predicting the next output. For example, in the sentence "The cat sat on the mat because it was tired", the word "it" should pay attention to "cat" rather than "mat". This ability to dynamically weight context is what makes attention powerful. We call this weight 'attention score'.

Each input token representation $z_t \in \mathbb{R}^d$ (embedding + positional encoding) is projected into three different vectors:

$$q_t = W_Q z_t, \quad k_t = W_K z_t, \quad v_t = W_V z_t,$$

where q_t is the *query*, k_t the *key*, and v_t the *value*, and $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$ are learned weight matrices.

The attention score between two tokens i and j is given by the scaled dot product between query q_i and key k_j:

$$\alpha_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}}.$$

These scores are normalized across all possible j using the softmax function:

$$a_{ij} = \frac{\exp(\alpha_{ij})}{\sum_{j'} \exp(\alpha_{ij'})}.$$

Finally, the output for position i is a weighted sum of the values:

$$\text{Attention}(q_i, K, V) = \sum_{j=1}^{n} a_{ij} v_j.$$

In matrix form, for a whole sequence:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

where Q, K, V are matrices stacking all queries, keys, and values.

Multi-Head Attention. Instead of computing one attention distribution, We use multiple parallel "heads":

$$\text{MHA}(Z) = \text{Concat}(h_1, h_2, \ldots, h_H)\, W_O,$$

where each head $h_h = \text{Attention}(Q_h, K_h, V_h)$ uses its own projection matrices. This allows the model to capture different types of relationships (e.g., syntactic vs. semantic).

Transformers The Transformer is a deep neural network built by stacking layers that combine attention and feed-forward processing.

Each layer consists of two main blocks:

1. **Multi-head self-attention**, which enables tokens to interact with one another.
2. **Feed-forward network (FFN)**, applied independently to each token.

Both blocks are wrapped with residual connections and layer normalization for stability.

At the top of the stack, the model produces contextualized token representations. A softmax output layer maps these into probabilities over the vocabulary:

$$p_\theta(x_{t+1} \mid x_{1:t}) = \text{softmax}(W_{\text{out}} z_t^{(L)}),$$

where $z_t^{(L)}$ is the final representation of token t after L layers.

This architecture enables Transformers to efficiently model long-range dependencies, scale to billions of parameters, and achieve state-of-the-art results across natural language tasks.

Large vs. Small Language Models Large Language Models (LLMs) and Small Language Models (SLMs) are based on the same Transformer architecture, but differ primarily in scale and deployment objectives. LLMs typically contain billions to hundreds of billions of parameters and are trained on massive datasets. Their size allows them to achieve very strong performance across a wide range of reasoning, coding, and language understanding tasks, but they demand enormous computational resources both during training and inference. In contrast, SLMs usually range from a few hundred million to a few billion parameters. While smaller, they are designed to be more efficient, easier to deploy on modest hardware, and increasingly capable thanks to higher-quality training data and improved training methods Lu et al. (2024). The distinction is thus less about the underlying architecture and more about scale, efficiency, and practical usability.

AI Agents AI agents can be seen as systems that use language models as their reasoning core, but extend their abilities by interacting with the external world. Instead of simply generating text, an agent can plan a sequence of actions, query databases, call APIs, or control software tools to solve complex tasks. For example, an AI agent powered by a language model could search for information online, summarize it, and then send an email with the results. The defining aspect of an AI agent is autonomy: it perceives an environment (through input or context), decides what to do next, and executes actions toward a goal. This transforms a language model from a passive predictor of text into an active problem solver that can operate in dynamic and interactive settings.

There are two principal types of AI agents commonly discussed in the literature:

- **Code Agents:** In this paradigm, the Language Model reasons about the task and interacts with the external world by generating executable code. This code is then executed within a controlled environment, and the resulting outputs guide the next reasoning steps.
- **Tool Agents:** Here, the Language Model is provided with descriptions of available tools. It reasons about which tools to use and interacts with the external world by generating structured tool requests, which are subsequently parsed and transformed into API calls or function executions.

3 LITERATURE REVIEW

During my internship, I conducted a literature review of research papers relevant to the central topic of my work: improving the reasoning and planning capabilities of AI agent orchestrators relying solely on Small Language Models (SLMs). This section provides a synthesis of the main insights drawn from that review.

3.1 Small Language models performance limit

There is a fundamental trade-off between model scale and reasoning performance, where smaller models are more efficient but often less capable on complex tasks.

A foundational insight in the study of language model scaling is that **performance, typically measured by validation or test loss, follows predictable trends governed by scaling laws**. These trends were rigorously quantified in Kaplan et al. (2020), which established key relationships between **model size**, **dataset size**, and **expected performance**. Specifically, Kaplan et al. (2020) formalized the expected loss $L(N, D)$ as a function of the number of model parameters N and the number of training tokens D:

$$L(N, D) = \left[\left(\frac{N_c}{N} \right)^{\alpha_N} + \left(\frac{D_c}{D} \right)^{\alpha_D} \right] \tag{1}$$

This formulation illustrates that both the number of parameters and the size of the training dataset must be **scaled in tandem**. If either N or D is held constant while the other increases, the test loss **plateaus**, as the fixed term becomes the bottleneck limiting

further performance improvements. Interestingly, Kaplan et al. (2020) also observed that **architectural details**—such as the number of layers, the depth-to-width ratio, and the dimensionality of the feedforward network (FFN)—have a minimal impact on overall test loss, suggesting that **overall model scale is a stronger predictor of performance than micro-architectural choices**.

In other words, a small model has inherent **performance limits**, regardless of efforts to expand the dataset or modify architectural design. Even though Hoffmann et al. (2022) proposed an alternative scaling law to optimize performance under a fixed compute budget, the same principle holds: if the model size is not increased, performance will plateau despite enlarging the training dataset. This insight directly informed the methodology of the present research. Rather than attempting to increase the performance of SLMs via additional data or architectural tweaks—which would be limited in effect, especially considering that most contemporary SLMs are already trained on **carefully curated, high-quality datasets** by teams of researchers and data scientists—we chose an **adaptive approach**.

Specifically, we designed **new reasoning and memory strategies tailored to SLMs**, to assess if **structural and algorithmic innovations** can compensate for limited model scale. By leveraging backward–forward reasoning and a tree-structured memory architecture, our approach aims to enable **effective tool use and multi-step problem solving** even when computationally lightweight models are employed.

3.2 Benchmarks for evaluating SLM reasoning and tool-use capability

Recent studies have evaluated language models on general-purpose benchmarks to assess their reasoning and tool-use capabilities. Common benchmarks include narrative understanding, such as HellaSwag Zellers et al. (2019), which tests the model's grasp of contextually plausible story continuations; common sense reasoning datasets like BoolQ Clark et al. (2019), OpenBookQA Mihaylov and Clark (2018), and CommonsenseQA Talmor et al. (2019), which evaluate the ability to answer questions requiring implicit background knowledge and logical connections; and problem solving and general knowledge datasets such as MMLU Hendrycks et al. (2021) and ARC Clark et al. (2018), which include multiple-choice questions across various disciplines, simulating the reasoning needed for academic and professional tasks. These datasets serve as valuable proxies for assessing a

model's ability to understand, reason, and respond intelligently in complex scenarios.

3.3 Influence of Training Datasets Quality

Even though Small Language Models (SLMs) are constrained by their limited number of parameters, and thus eventually plateau in performance, there remains substantial room for improvement through careful dataset curation and extended training. The choice and quality of training data directly influence model capabilities. In fact, recent work has shown that new SLM releases consistently surpass their predecessors, sometimes approaching the performance of previous generation larger models Lu et al. (2024), largely due to higher-quality training datasets.

Earlier generations of SLMs were primarily trained on large-scale web corpora such as Common Crawl Wenzek et al. (2020), often preprocessed with pipelines like `ccNet` Wenzek et al. (2020), which perform deduplication, language identification, and heuristic filtering. As the field matured, more refined datasets emerged: for example, **FineWeb-Edu** Xu et al. (2023) leverages larger LLMs to automatically filter and rank educationally valuable content, while code-oriented datasets such as **StarCoder** Li et al. (2023a) were added to improve reasoning and problem-solving. Lu et al. (2024) confirms that reasoning ability, factuality, and coding performance benefit disproportionately from these curated corpora compared to sheer data volume. Moreover, domain-specific fine-tuning has proven critical in specialized environments: for instance, biomedical models fine-tuned on Chain-of-Thought (CoT) datasets demonstrate significant gains in clinical reasoning and diagnostic tasks Kim et al. (2024). During my internship, due to time constraints, I did not curate new datasets, but I strongly recommend applying our method in conjunction with domain-adapted or high-quality training corpora, as dataset design remains a key lever for improving SLM performance.

3.4 Examples of Recent SLMs

SmolLM2. The SmolLM2 family, introduced by Ben Allal et al. (2025), is a set of decoder-only Transformer models released in sizes of 135M, 360M, and 1.7B parameters. All variants follow a LLaMA-style architecture Touvron et al. (2023) with rotary positional embeddings (RoPE) Su et al. (2021) and SwiGLU activations Shazeer (2020). Grouped Query Attention (GQA) Ainslie et al. (2023) is used only in the smaller 135M

and 360M variants to reduce memory footprint, while the 1.7B model retains standard multi-head attention. Training followed a multi-stage strategy over roughly 11 trillion tokens: beginning with large-scale web data, then progressively annealing in high-quality math and code datasets such as FineMath and Stack-Edu, and finally instruction tuning. For post-training, the team introduced SmolTalk, a new instruction-following dataset that combines opensource datasets, and curated math and code data. Alignment was performed with Direct Preference Optimization (DPO) Rafailov et al. (2023).

Dataset curation played a central role: filtered Common Crawl–derived web data (FineWeb-Edu, DCLM) was mixed with educational corpora, Stack-Edu for code, FineMath for mathematics, and synthetic data such as Cosmopedia. Ablation studies confirmed that curriculum-style training and late-stage upsampling of reasoning-focused datasets substantially boosted performance. Evaluation shows that SmolLM2 achieves strong results relative to its size: on GSM8K it reaches above 30% (5-shot), on HumanEval around 22–28%, and on MMLU-Pro it surpasses Qwen2.5-1.5B while remaining competitive with larger models. The models also natively support an extended context length of 8k tokens.

Qwen2.5. The *Qwen2.5* series Yang et al. (2024) extends Alibaba's line of multilingual LLMs, released in sizes from 0.5B up to 72B dense parameters. For small-model research, the most relevant are the 0.5B, 1.5B, and 3B versions. Architecturally, Qwen2.5 maintains a Transformer decoder-only design with Grouped Query Attention (GQA) Ainslie et al. (2023) for efficient KV cache usage, SwiGLU activations Shazeer (2020), Rotary Positional Embeddings (RoPE with extended base frequency) Su et al. (2021), QKV bias Su (2023), and RMSNorm with pre-normalization Zhang and Sennrich (2019).

Pretraining data was expanded substantially compared to Qwen2—from 7T to **18T tokens**—with careful filtering, domain balancing, and integration of higher-quality math and code datasets (including Qwen2.5-Math and Qwen2.5-Coder) alongside synthetic data. Long-context ability was strengthened with staged context extension up to 32k tokens for all open-weight dense models.

Post-training involved a multi-phase pipeline: supervised finetuning on over 1M examples (covering math, code, instruction following, structured data, reasoning, and multilingual data), followed by reinforcement learning via **Direct Preference Optimization (DPO)** Rafailov et al. (2023) in the offline phase and **Group Relative Policy Optimization (GRPO)** Shao et al. (2024) in the online phase.

Evaluation shows notable gains for the smaller models. **Qwen2.5-0.5B** surpasses its Qwen2 predecessor across general, math, and code tasks; **Qwen2.5-1.5B** achieves ∼55 on GSM8K and >60 on MBPP; and **Qwen2.5-3B** reaches ∼66 on MMLU and ∼74 on HumanEval, rivaling or exceeding other open models up to 4B parameters. Long-context evaluation confirms that even the smallest Qwen2.5 models maintain strong retrieval accuracy up to 32k tokens, though near-perfect "needle-in-a-haystack" performance is only reported for larger models. During our internship, we will use the Qwen2.5-1.5B-Instruct model for its long-context capabilities, which are essential for maintaining extended memory in linear paradigms such as ReAct. This model represents the smallest viable option that still provides a reasonable capacity for instruction-following tasks.

3.5 Mini-Survey on AI agents reasoning paradigms and architectural frameworks

3.5.1 Prompting Paradigms

ReAct ReAct (Reasoning and Acting) Yao et al. (2023b) is a prompting framework that guides a language model to interleave reasoning steps ("Thoughts") with external actions ("Acts") during inference 1. At each time step, the model is prompted to produce outputs in a defined sequence:

`Thought: ... Act: ... Observation: ...`

The "Thought" is an internal reasoning step expressed in natural language. The "Act" is an external operation such as a tool call, web search, or environment action. The "Observation" records the resulting feedback from the environment. After each step, the generated sequence (`Thought`, `Act`, `Observation`) is appended to the prompt, forming a *linear memory* that accumulates the full trajectory of the interaction. This growing context is then fed back to the model, prompting it to generate the next step until the task is solved.

To implement this, the model is guided through *few-shot prompting*. The prompt includes a handful of demonstration trajectories—short examples consisting of Thoughts, Actions, and Observations—that show how to solve similar tasks. These examples teach the model both the desired output format and the reasoning style needed for the domain. During inference, the model follows this pattern autonomously: it reasons over the context with a Thought, proposes an Act, observes the result, and then repeats the cycle.

Thoughts can serve multiple purposes, such as breaking down a complex goal into

steps, planning sequences of future actions, injecting commonsense knowledge, extracting key details from observations, monitoring progress, or adapting a plan when unexpected outcomes occur. While these Thoughts have no direct effect on the environment, they enrich the model's context and guide subsequent actions more effectively.

On knowledge-intensive tasks like HotpotQA and FEVER, ReAct reduces hallucinations compared to chain-of-thought prompting by grounding reasoning in external observations, achieving 27.4 EM and 60.9% accuracy respectively with PaLM-540B. On interactive decision-making benchmarks such as ALFWorld and WebShop, ReAct reaches success rates of up to 71% and 40%, surpassing imitation and reinforcement learning baselines by absolute margins of 34% and 10% respectively, with just one or two in-context examples.

During the internship we implemented this paradigm alongside the Act-only baseline [1] to compare against our novel method, and to evaluate whether such improvements remain valid when using small language models under 2B parameters.

PlanAct Plan-and-Solve prompting—commonly referred to as PlanAct— Li et al. (2023b) is a zero-shot strategy that structures reasoning into two stages: (1) *Plan*, where the model outlines sub-goals or subtasks; and (2) *Solve*, where it executes them sequentially. By first breaking complex tasks into manageable steps, PlanAct significantly improves reasoning performance over straightforward zero-shot chain-of-thought ("Let's think step by step") strategies. In evaluations across ten datasets, PlanAct consistently surpasses zero-shot CoT and approaches the effectiveness of few-shot CoT prompting—even rivaling eight-shot CoT performance on mathematical reasoning tasks.

ReWOO ReWOO (Reasoning Without Observation) Li et al. (2023c) separates the reasoning phase from external-action feedback to reduce token usage and simplify computation. It divides the reasoning process into three modular components: a Planner (constructs a reasoning blueprint), a Worker (executes the plan internally), and a Solver (generates the final answer). This modular design notably lowers the number of tool invocations and tokens required while maintaining task performance and offers a streamlined architecture for efficient multi-step reasoning tasks.

[1] The Act-only baseline removes all reasoning steps ("Thoughts") from the prompt and only allows the model to generate external actions, without interleaving them with reasoning.

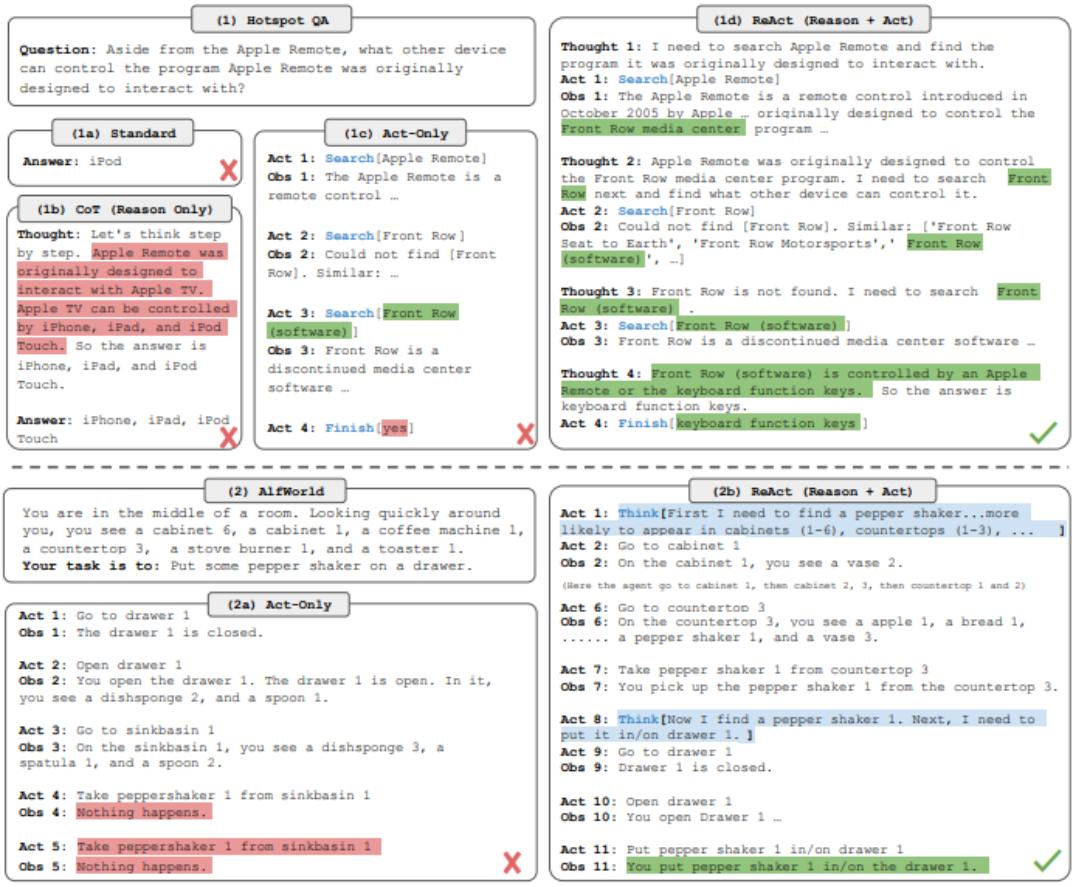


Figure 1: (1) Comparison of 4 prompting methods, (a) Standard, (b) Chain-of-thought (CoT, Reason Only), (c) Act-only, and (d) ReAct (Reason+Act), solving a HotpotQA (Yang et al., 2018) question. (2) Comparison of (a) Act-only and (b) ReAct prompting to solve an AlfWorld (Shridhar et al., 2020b) game. In both domains, we omit in-context examples in the prompt, and only show task solving trajectories generated by the model (Act, Thought) and the environment (Obs).

Figure 1: ReAct Paradigm

3.5.2 Papers Inspiring My New Method

Tree of Thoughts The *Tree of Thoughts* (ToT) Yao et al. (2023a) framework extends linear chain-of-thought prompting into a structured exploration of multiple reasoning paths. Rather than committing to a single trajectory, the model generates several "thought" candidates at each step, evaluates their potential, and continues reasoning along the most promising branches—while allowing for backtracking when dead ends are reached. On tasks such as the "Game of 24," this approach elevates performance dramatically: GPT-4 with traditional chain-of-thought solves only 4% of cases, whereas ToT achieves 74% success using breadth-first search with five branches per step. This method inspires my approach of structured reasoning that can selectively branch, evaluate, and prune paths to improve inference robustness.

LAMBADA *LAMBADA* Guo et al. (2023) introduces a backward-chaining strategy, reversing the typical forward proof search by starting from the conclusion and working backwards toward supporting premises. The model is prompted to decompose reasoning into submodules—each implemented via few-shot prompting—thus guiding stepwise inference toward an explicit logical structure. Compared to forward reasoning (e.g., chain-of-thought), LAMBADA achieves significant gains on deep logical reasoning benchmarks where longer proof chains are needed. This backward structure aligns with my method's goal of integrating a goal-oriented, explanation-first reasoning component.

Monte Carlo Tree Search (MCTS) Monte Carlo Tree Search (MCTS) is a heuristic search algorithm that incrementally builds a search tree to make decisions in large or complex state spaces. Starting from a root node, it recursively selects child nodes using a policy that balances exploitation (favoring nodes with high observed rewards) and exploration (favoring less-visited nodes), typically via the Upper Confidence Bound for Trees (UCT). When a leaf node is reached, the tree is expanded with possible actions, and a simulation or "rollout" is performed to estimate the outcome. The resulting reward is then backpropagated to update value estimates for all ancestor nodes. Mathematically, MCTS estimates the action-value function $Q(s, a)$ for each state-action pair, and as the number of simulations grows, the selection policy converges to the optimal action. This framework is widely used in games, AI planning, and reasoning tasks where exhaustive search is infeasible. It has been successfully adapted to LLM-based reasoning in frameworks such as **RAP (Reasoning via Planning)** Hao et al. (2023). RAP leverages MCTS to explore multiple reasoning paths in a structured manner, selecting the most promising branches while pruning less fruitful ones. This approach allows the model to iteratively refine its reasoning and decision-making, improving performance on complex, multi-step tasks. By integrating MCTS with retrieval and planning mechanisms, RAP has demonstrated significant gains on knowledge-intensive benchmarks, particularly for small language models, enabling them to approach the reasoning capabilities of much larger models. This structured search strategy directly inspires the tree-based reasoning and selective refinement mechanisms in our method.

Self-Questioning Prompting A recent method termed **Introspective Growth** trains small language models to improve their understanding by prompting them to gener-

ate—and then answer—their own clarifying questions Wu et al. (2025). In this approach, the model is prompted to pose questions about the input (e.g., "Why does this distinction matter?"), attempts to answer them, and uses its own responses to activate latent knowledge that might otherwise remain unused. Experiments on a challenging benchmark of technical patent document comparisons showed that models trained in this way outperformed standard baselines; the performance further improved when combined with retrieval from external scientific texts. Notably, self-questioning was more effective than standard chain-of-thought prompting at enhancing the model's comprehension of nuanced, technical distinctions, and smaller models sometimes generated more open-ended, fundamental questions that aided understanding more than larger models did.

4 PROBLEM STATEMENT

4.1 Problem Statement

The recent rise of AI agents capable of interacting with complex environments through external tools (APIs, search engines, visualization software, domain-specific assistants, etc.) has opened the way to more flexible and autonomous systems. These so-called *tooled agents* are characterized by their ability to select and combine multiple tools to perform diverse tasks, ranging from simple action execution to more advanced processes involving reasoning, planning, or exploration.

Despite the proliferation of frameworks and orchestration paradigms (e.g., ReAct 3.5.1, PlanAct 3.5.1, RewOO 3.5.1), there is currently no clear or widely accepted classification of the types of problems these agents face, in relation to the cognitive and decision-making requirements they entail. As a consequence, engineers often make design choices arbitrarily or based on intuition when selecting an agent architecture or orchestration paradigm. This results either in oversized, inefficient systems, or in insufficient architectures for tasks requiring high autonomy or reasoning capabilities.

Current research on AI agents remains fragmented. Existing benchmarks such as BabyBench, ToolBench, or OfficeBench focus on specific scenarios but fail to provide a general framework for understanding the nature of the underlying problems.

Research Gap. What is missing today is:

- A rigorous and operational taxonomy of problems encountered by tooled AI agents in realistic environments.
- Formal recommendations linking this taxonomy to existing orchestration paradigms.
- Simple and targeted benchmarks to empirically validate the adequacy between problem nature and architectural choices.

This project is situated precisely within this scientific and technical gap.

Industrial Context. The research is motivated by a concrete need arising from the *Genysis* project, which aims to design a **Small Action Model (SAM)** capable of planning and orchestrating the behavior of multiple AI agents in a defined application environment. Building such a SAM requires a precise understanding of the different natures of agentic tasks, as well as the corresponding orchestration and reasoning requirements. The outcomes of this research will therefore:

- Guide Genysis engineers in selecting and designing the SAM according to the agents and tasks involved.
- Provide a generic framework, useful beyond Genysis, to help any team designing multi-agent tooled systems better connect problem nature, reasoning requirements, and appropriate orchestration paradigms.

Global Research Questions. The project is guided by the following questions:

> *Can we establish an operational taxonomy of tooled AI agent problems to guide the design of orchestrators? How can such a taxonomy be exploited to design new orchestration paradigms adapted to the specificities and constraints of each type of problem?*

In the context of Genysis, this can be reformulated as:

> *How can we choose the most appropriate SAM paradigm to coordinate multiple agents, given technical constraints and the complexity of the scenarios?*

Mathematical Formulation. The problems of tooled AI agents can be formalized using the framework of **Partially Observable Markov Decision Processes (POMDPs)**:

- **Latent states (S):** The true system state is latent and not directly observable. Due to the complexity of the environments and tools, these states are difficult to explicitly define.
- **Observation space (O):** The agent perceives sequential observations derived from tool outputs over a time horizon. Formally, O is an infinite and structured space containing all possible concatenations of tool outputs, which constitute the only source of information for the agent.
- **Action space (A):** Unlike classical POMDPs where A is finite, here A is infinite. Each action corresponds to the use of a tool with specific input parameters, which may vary continuously or discretely, generating unbounded possibilities.
- **Policy (π):** A policy maps observation histories (or derived beliefs) to actions, i.e., the selection of tools and parameters. Reasoning is considered an internal mechanism of the policy rather than an action itself.

Scope. For simplicity, the first experiments will exclude code-generating agents, though orchestrators may still have access to an external code-generation tool. Furthermore, we will restrict ourselves to a single orchestrator agent, considering other agents either as inputs, observations, or tools to be selected.

4.2 Proposed Taxonomies

After analyzing multiple AI agent benchmarks and comparing the types of challenges they present, we propose a taxonomy that classifies the complexity of tooled AI agent orchestration problems along several key dimensions:

- **Solution Path Complexity:** Evaluates the difficulty in finding the sequence of actions and tools required to solve the problem. High complexity arises when intermediate steps are not obvious, or tools are highly granular and can be used in multiple ways.

- **Reasoning Complexity Between Actions:** Concerns the depth and sophistication of reasoning needed to determine the next action after an observation. High complexity implies transforming outputs from prior actions using logical reasoning to generate inputs for subsequent actions. For instance, in the GTA benchmark, the orchestrator may need to select the largest element from a list and feed it into another tool. Low complexity occurs when actions are independent, as in ToolBench.
- **Tool Selection Complexity:** Measures the difficulty of selecting the appropriate tool from a potentially large and varied set of available tools.
- **Clarity of Inputs, Tools, and Instructions:** Assesses how clear and complete the information provided to the agent is regarding the task, tools, and input data.
- **Number of Steps:** Represents the typical length of an action sequence required to accomplish a task.
- **Number of Tools:** Indicates the quantity of distinct tools an agent may use for a given task.

Tables 1 and 2 applies this taxonomy to several open-source benchmarks (ToolBench OpenBMB (2024), GTA Bench Wang et al. (2024a), ComplexFunc Bench Zhong et al. (2025), Office Bench Wang et al. (2024c), MTU Bench Wang et al. (2024b)), assigning a relative complexity level (Low, Medium, High) for each dimension. This characterization clarifies the types of challenges each benchmark presents for AI agent orchestrators.

Table 1: Benchmark assessment: Solution Path, Reasoning, Tool Selection

Benchmark	**Solution Path**	**Reasoning**	**Tool Selection**
ToolBench	Low	Low	Low–High
GTA Bench	Medium–High	High	Low–Medium
ComplexFunc Bench	High	High	Low–Medium
Office Bench	Medium	Medium–High	Medium
MTU Bench	Low	Low–Medium	Low–Medium

4.3 Our Research Question

Given time constraints during the internship, it was necessary to focus our research on a more specific and tractable problem. Guided by the proposed taxonomy, we selected two key dimensions as the primary focus of this work:

Table 2: Benchmark assessment: Input Clarity, Number of Steps, Number of Tools

Benchmark	**Input/Instruction Clarity**	**Number of Steps**	**Number of Tools**
ToolBench	High (tools)	Low	Medium–High
GTA Bench	High (inputs)	Medium	Low
ComplexFunc Bench	Low–Medium	High	Medium
Office Bench	Low–High	Medium–High	High
MTU Bench	High (instructions)	Low	Low

- **Solution Path Complexity:** The difficulty in identifying the correct sequence of actions and tool usages needed to achieve a goal.
- **Reasoning Complexity Between Actions:** The depth of reasoning required to determine the next action based on the outputs of previous actions.

To isolate the impact of these two dimensions, we deliberately chose an environment and dataset where the remaining dimensions (tool selection, clarity of inputs, number of steps, and number of tools) are kept low in complexity. This ensures that the experimental results are not biased by unrelated factors and that any observed performance differences are attributable to the chosen dimensions.

By narrowing the scope, we aim to identify which orchestration paradigms are most effective for agentic problems that combine a complex solution path with non-trivial reasoning between actions. The primary research question guiding this study can thus be formulated as:

> *What is the best orchestration paradigm to solve agentic problems characterized by high solution path complexity and high reasoning complexity between actions?*

This targeted focus also opens opportunities for future research. Subsequent studies could explore interactions among multiple high-complexity dimensions, examining whether conflicts or trade-offs emerge when an environment requires both sophisticated reasoning, tool selection, and input disambiguation. Such investigations may reveal new challenges in multi-dimensional agentic problem solving and inspire novel orchestration strategies beyond the scope of the present work.

5 DATASET AND ENVIRONMENT

5.1 The GTA Benchmark and Our Dataset Enhancement

Overview of the GTA Benchmark The GTA (General Tool Agents) benchmark Wang et al. (2024a) is a comprehensive evaluation framework designed to assess the tool-use capabilities of large language model (LLM)-based agents in real-world scenarios. (Figure 2) It comprises two primary components:

- **Real User Queries:** The dataset includes 229 human-written queries that present simple real-world objectives. These queries are implicitly tool- and step-agnostic, requiring the LLM to reason about the appropriate tools and plan the sequence of actions ?.
- **Real Multimodal Inputs:** Each query is accompanied by authentic image files, such as spatial scenes, web page screenshots, tables, code snippets, and printed/handwritten materials, used as the query contexts to closely align with real-world scenarios.

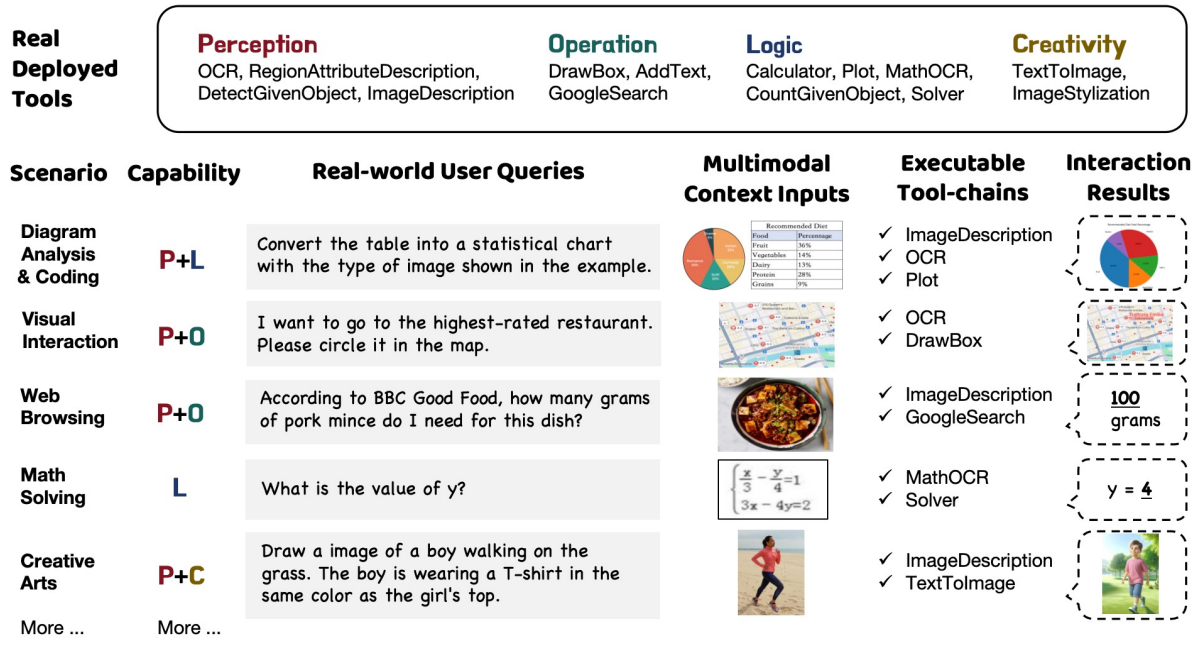


Figure 2: GTA Dataset

These features make the GTA benchmark an ideal candidate for evaluating agents in complex, multimodal environments, aligning well with our research focus on *Solution Path Complexity* and *Reasoning Complexity Between Actions*.

Challenges with Input Clarity Despite its strengths, the GTA benchmark presents challenges in input clarity. The multimodal inputs, while realistic, can sometimes be ambiguous or lack sufficient context, potentially introducing noise into the evaluation process. This ambiguity can affect the agent's performance, especially in tasks requiring precise interpretation of inputs.

Enhancement: Clear GTA Dataset To address this issue, we have developed an enhanced version of the GTA dataset, termed the *Clear GTA Dataset*. In this revised dataset, we have augmented the multimodal inputs with detailed textual descriptions of the images. These descriptions provide explicit context and clarify any ambiguities present in the original inputs. By doing so, we aim to reduce the complexity associated with input interpretation, allowing the agent to focus more effectively on the orchestration and reasoning aspects of the tasks.

Justification for Dataset Selection We selected the GTA benchmark as the foundation for our research because it naturally embodies tasks requiring high *Solution Path Complexity* and *Reasoning Complexity Between Actions*. Each task often involves multiple sequential steps, where the agent must determine not only which tools to use, but also the correct order and combination of actions to reach the goal. For example, in a task where the agent must extract numerical data from a table and feed it into a plotting tool, the model must first parse the table, select relevant rows and columns, compute aggregates if necessary, and finally provide the correctly formatted input to the plotting tool. Each of these intermediate steps demands reasoning about the outputs of previous actions to generate valid inputs for subsequent tools. The model must plan multi-step sequences and reason over intermediate outputs, such as computing summaries, performing conditional logic, or chaining outputs across tools. For instance, in a task where multiple images need to be interpreted, the agent must combine extracted information from each image, reason about their interdependencies, and decide which tool or action to invoke next.

the Clear GTA Dataset keeps other dimensions of the taxonomy low in complexity as desired, such as *Tool Selection Complexity*, *Number of Tools*, *Number of Steps*, and *Clarity of Inputs*. The benchmark provides a limited set of tools (3 to 4 tools in average), the number of steps is moderate (maximum 8 steps), and inputs are made unambiguous

via textual descriptions.

5.2 Testing Environment

To evaluate agentic reasoning and orchestration paradigms, we constructed a controlled testing environment simulating interactions between the agent and multiple tools. Each task in the environment specifies a task instruction, available tools, initial inputs, expected outputs, and valid sequences of tool invocations.

When the agent invokes a tool, the system checks whether the tool exists, verifies required arguments, and ensures that argument values are compatible with expected formats. It then checks whether the argument values of the tool request correspond to a valid invocation from the test dataset, using either fuzzy matching, exact matching or "LLM matching" [2] depending on the argument type. If the argument values do not match, we faced two options for the environment's response: either return a signal indicating that the argument or tool does not lead to the correct answer, or provide random outputs. While the latter is more realistic, it introduces additional complexity by requiring the agent to identify the source of the error. We opted for the former approach to maintain simplicity and focus on a single dimension of complexity, leaving exploration of other dimensions for future work.

This approach has inherent limitations. The static environment does not fully capture the dynamic interactions present in real-world tool-augmented agents. Consequently, while the environment is well-suited for testing multi-step reasoning and solution path complexity, it may not fully reflect performance in dynamic or open-ended scenarios. Despite these limitations, the setup still provides a practical, controlled platform aligned with our research focus on evaluating orchestration paradigms in tasks with high reasoning and path complexity.

6 OUR NOVEL METHOD: MINIMIND AGENT

6.1 Overview of the Method

Definitions: First let us define two fundamental reasoning approaches:

[2] LLM matching refers to asking an LLM to decide whether the requested argument value matches a valid argument value.

- **Forward reasoning**, also known as forward chaining, begins with known facts and applies inference rules to extract more facts until a goal is reached. This method is data-driven and is commonly used in systems where the objective is to derive conclusions from a set of premises. In our context, Forward reasoning starts with the current state of the environment and known facts. The agent iteratively applies available actions or tool calls to generate new states, progressively moving toward a solution. It is exploratory: the agent "pushes forward" from what it knows, checking at each step whether the goal has been reached.

- **Backward reasoning**, or backward chaining, starts with a goal and works backward to determine what facts must be true to achieve that goal. This approach is hypothesis-driven and is often employed in scenarios where the objective is to determine the steps leading to a specific conclusion. In the context of AI Agents, Backward reasoning starts with the desired goal and works backward to identify the prerequisites or actions needed to achieve it. The agent decomposes the task into subtasks recursively until reaching actions that can be executed directly in the environment.

MiniMind Agent Our proposed method introduces the **MiniMind Agent** paradigm, designed to handle complex agentic tasks that require multiple tool calls and reasoning steps. Its core idea is to move beyond linear reasoning loops by structuring the problem-solving process as a hierarchical tree of reasoning and action.

In this tree, the task is represented as a root node that decomposes into subtasks, represented by child nodes. Child nodes belong to one of four types depending on whether they involve reasoning or tool use, and whether they operate backward or forward: *BackwardReasoningStep*, *BackwardToolStep*, *ForwardReasoningStep*, or *ForwardToolStep*. While the first three types can act as either intermediate nodes or leaves, *ForwardToolStep* nodes serve only as leaves. These are reserved for exploratory execution when deterministic backward reasoning cannot produce a solution. More detailed discussion of these node types is provided in subsection 6.2.

A central feature of MiniMind Agent is its ability to bridge backward reasoning with forward exploration in a controlled, tree-based memory. When backward reasoning reaches an impasse, the system prompts an SLM to perform forward exploration,

then evaluates whether the resulting output contains enough information to resolve the parent subtask. This process helps afterwards to push the reasoning even further by using other prompts to search for what is missing to plan a solution for the blocking subtask. This mechanism enables the agent to continue reasoning even when the solution path is not obvious. The tree memory further helps by preserving context across subtasks while allowing the system to selectively include only the most relevant information in prompts, reducing context length and bias while making prompts more focused.

MiniMind Agent also builds on several existing paradigms. From **ReAct** Yao et al. (2023b), it inherits the integration of reasoning and action, extending it from a linear chain into a tree structure. From **Tree of Thoughts** Yao et al. (2023a) and **LAMBADA** Guo et al. (2023), it incorporates hierarchical decomposition and intermediate subtask generation. Finally, it draws on self-reflective approaches Wu et al. (2025), enabling nodes to evaluate the sufficiency of available information and to spawn additional subtasks when required.

This design explicitly addresses the challenges of complex reasoning paths and intermediate processing steps. Unlike ReAct's linear reasoning-action loop, MiniMind Agent supports branching through its tree-based structure, combining backward chaining with forward exploration for greater robustness. The hierarchy also decomposes the task into subtasks and cleanly separates reasoning steps from tool use steps, improving both interpretability and reliability in multi-step problem-solving.

6.2 Design Choices

High-Level Design At the core of the MiniMind Agent lies its *tree memory*, which is organized according to the Composite design pattern. This tree memory represents the reasoning process of the agent as a hierarchy of interconnected elements. Each element of the tree is called a `Step`. The term *Step* is used because each node in the tree represents one step toward completing the overall task: it is responsible for reasoning about how to solve a specific subtask, and once ready, it executes this subtask and produces an output that its parent node can use. In this way, solving the global task is achieved step by step, by recursively solving subtasks and combining their results.

The abstract class `Step` plays the role of the *Component* in the Composite pattern. It is never instantiated directly; instead, it defines the common contract that every el-

ement of the tree must follow. Each node, regardless of its concrete type, possesses a *subtask* attribute that specifies the problem it is trying to solve, a *status* attribute that records its progress (e.g., pending, success, fail), and a set of methods such as `check_ready_to_execute` and `execute`. The first method returns a boolean determining whether the node has enough information to be able to solve its subtask. The second method returns the solution of the subtask that contributes to its parent's reasoning process.

In this structure, a node can act as either a *Leaf* or a *Composite*. Leaf nodes are directly executable steps that do not depend on other subtasks. Composite nodes, in contrast, can contain other steps as children, relying on them to provide the necessary outputs before they themselves can execute. This uniform interface allows the planner to treat all nodes consistently, whether they are simple leaves or complex composites.

Beyond this common interface, there exist different types of nodes, each with its own way of reasoning and its own way of executing its subtask. Each type corresponds to a concrete class that inherits from the abstract `Step` class and provides its own implementation of the methods `check_ready_to_execute` and `execute`. This ensures that, although all nodes share the same contract, they can behave differently depending on their specific role in the reasoning process.

These node types can be understood along two main dimensions. The first distinction is between *Forward* and *Backward* nodes. Forward nodes are always ready to execute: they do not generate child subtasks but directly attempt to advance toward solving the overall problem using forward reasoning. Backward nodes, by contrast, may generate child steps: they work by recursively decomposing their subtask into smaller subtasks that must be completed before they themselves can execute. Backward nodes first ensure that they possess sufficient information to complete their subtask. If the available information is insufficient, they generate new subtasks to obtain the necessary data, which results in the creation of additional child nodes. This mechanism embodies the process of backward reasoning. To accomplish this, multiple Small Language Model (SLM) calls are employed with carefully designed, targeted prompts that guide the reasoning process deeper, enabling the node to iteratively search for and acquire the precise information required to generate new subtasks.

The second distinction is between *Tool* nodes and *Reasoning* nodes. This distinction

lies in the nature of their subtasks and whether their execution requires a tool call or relies solely on an internal reasoning process. Tool nodes have subtasks that involve executing external tools to produce their output, whereas Reasoning nodes have subtasks that consist of performing internal reasoning instructions that do not require any tool. These reasoning instructions can include tasks such as extracting information from a text or identifying the maximum element in a list. Reasoning nodes execute these instructions by invoking a Small Language Model (SLM), which processes the instruction and generates an output usable by the parent node. In contrast, Tool nodes execute their subtasks by generating a structured JSON tool request containing the tool name and the values of its arguments, before receiving the result of their request from the environment.

By combining these two dimensions, four main types of nodes emerge: Forward Tool Steps, Backward Tool Steps, Forward Reasoning Steps, and Backward Reasoning Steps. Each type represents a different strategy for solving subtasks within the tree memory, and together they provide the flexibility needed for the agent to orchestrate complex reasoning and action. Each of these four types of nodes is implemented as a separate class that inherits from the abstract `Step` class. While all of these classes adhere to the common interface defined by `Step`, each implements the `check_ready_to_execute` and `execute` methods in a manner specific to its role. For instance, a Forward node always reports that it is ready to execute, whereas a Backward node may first generate and wait for the completion of child nodes before it is considered ready. Similarly, Reasoning nodes execute their internal reasoning instructions via a Small Language Model, while Tool nodes generate structured tool requests to invoke external tools. In addition to the common attributes defined in `Step` such as `task`, `status`, and `children`, each subclass can define additional attributes that are specific to its execution strategy, the type of tool it uses, or the nature of its reasoning instruction. The figure 3 presents a UML diagram of the tree memory.

The `Planner` class is responsible for constructing, managing, and traversing the tree of `Step` nodes. It determines which node is currently the *head* of the tree—the node that is ready to be executed—and orchestrates the execution of that node when appropriate. The planner also handles the traversal logic, moving to child nodes or returning to parent nodes based on execution results, and ensures that the overall reasoning process proceeds in a structured, step-by-step manner. Additionally, the planner can monitor the depth of

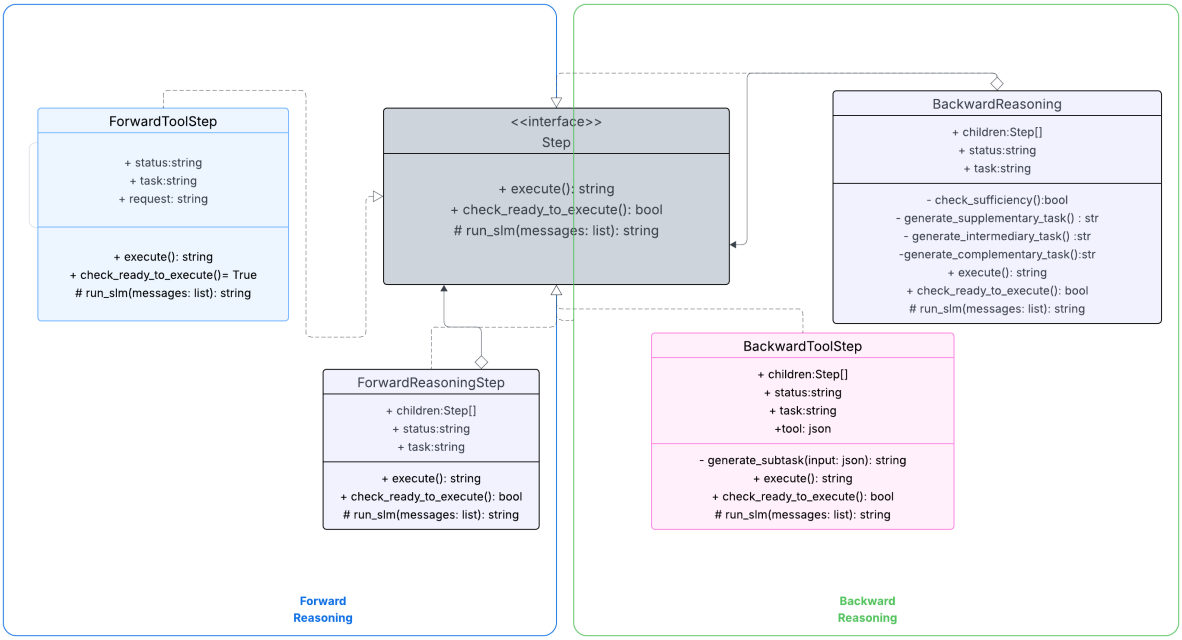


Figure 3: UML Diagram of the MiniMind Agent

the tree to prevent infinite recursion or excessive branching.

The `MiniMindAgent` class serves as the main agent interface, integrating the tree memory and the planner with the environment in which the agent operates. Each time the planner executes a node, the agent takes the resulting output and applies it in the environment, then collects the observation produced. The agent also manages the overall control flow, including deciding when to stop the reasoning process, updating the planner's head status, and adjusting planner behavior based on the observations received from the environment.

6.3 Detailed Flow: From Task to Solution

The execution of the **MiniMind Agent** begins with the root task, which is always instantiated as a *BackwardToolStep*. This root node represents the overall task to be solved and initiates the reasoning process.

The execution proceeds in three main phases:

Phase 1: Backward Chaining and Subtask Generation. The root *BackwardToolStep* first checks whether the inputs required by its associated tool are available in the environment. For each input that is missing or requires computation, a child node is generated as a *BackwardReasoningStep*, with the subtask of producing the value of that specific argument. Each *BackwardReasoningStep* first determines, using a targeted SLM

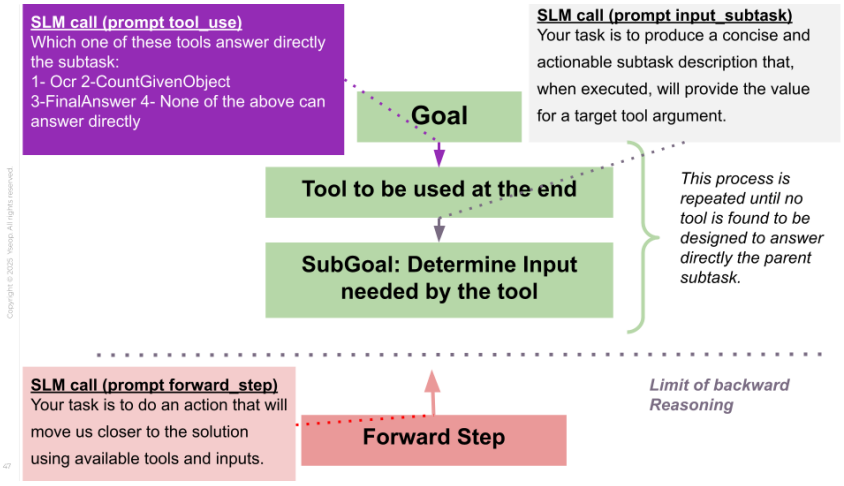
prompt, whether there exists a tool capable of providing the required value. If such a tool exists, the node creates a child *BackwardToolStep* for that tool, and the same process repeats recursively. If no tool can produce the required input, a *ForwardStep* is generated as a child node. This forward node executes exploratory actions by generating tool calls to produce output that can unblock the backward reasoning process. Tool selection, determination of whether an argument requires a subtask, and the generation of these subtasks are all performed via dedicated SLM calls with targeted prompts, ensuring precise and context-specific reasoning at each step. Figure 4a illustrates this first phase of the process.

Phase 2: Evaluation of Forward Output. Once a forward node has been executed, the backward node that was previously blocked uses the forward output to reason further. A dedicated SLM prompt is used to classify the forward output according to its utility for the backward task: (i) the output is sufficient to complete the reasoning, (ii) the output contains relevant information but other information is missing, (iii) the output contains relevant information but it is difficult to extract without additional input, or (iv) the output is totally irrelevant. This classification guides the subsequent steps in the reasoning tree. Figure 4b illustrates this second phase of the process.

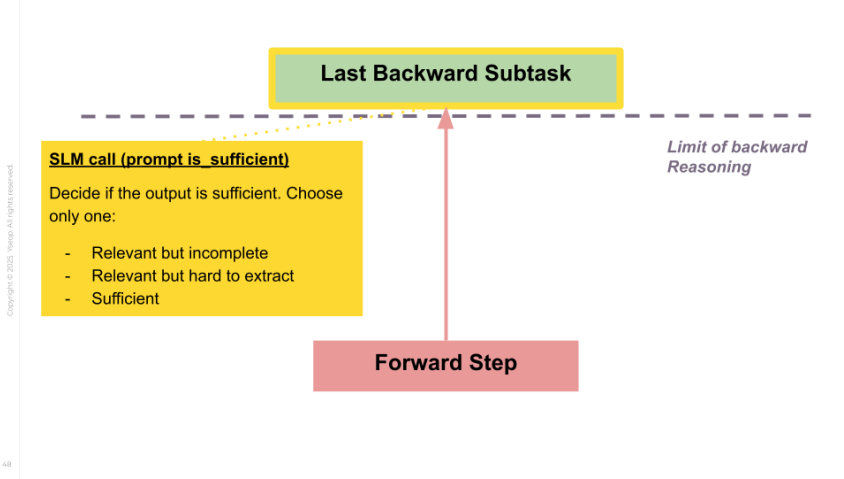
Phase 3: Bridging Forward and Backward Reasoning. Depending on the classification of the forward output, new nodes are created to integrate forward exploration into backward chaining. If the output is missing information, a new *BackwardReasoningStep* is generated to search for the required data. If the output contains relevant information that is difficult to extract, an intermediary node is created between the forward step and the blocked backward node. The role of this intermediary node is to extract the relevant information from the forward output. Additionally, this intermediary node may itself have child *BackwardReasoningStep* nodes to retrieve missing information needed for the extraction. All decisions in this phase—including whether to generate supplementary nodes and how to structure their subtasks—are guided by SLM calls with specialized prompts. Figure 4c illustrates this third phase of the process.

Throughout all three phases, the reasoning tree dynamically grows, alternating between backward and forward steps as necessary. Each SLM call is designed to provide concise, context-specific instructions, allowing the MiniMind Agent to iteratively decompose the task into executable subtasks, propagate observations back up the tree, and

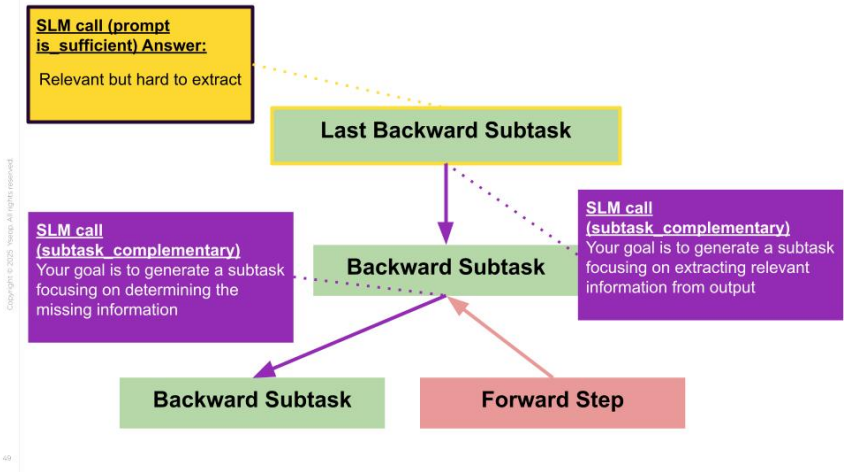
ultimately converge on a solution.



(a) Phase 1: Backward chaining and Forward Exploration



(b) Phase 2: Deep Reasoning to bridge Backward Chaining with Forward Output



(c) Phase 3: Bridging with Intermediate Backward Steps

Figure 4: MiniMind Agent Paradigm: Overview of backward chaining, forward exploration, and intermediate steps.

7 RESULTS

In this section, we present the results of our experiments and observations.

7.1 Experiment Overview

We implemented three agent paradigms—OnlyAct, ReAct, and MiniMindAgent—as well as the environment, all developed from scratch in Python. For language model integration, we relied solely on the transformers library to use open-source small language models. Importantly, we did not employ any agent framework libraries, in order to maintain full control over the experimental setup and ensure simplicity of implementation, avoiding additional optimized techniques that might bias our conclusions.

We evaluated two baseline paradigms on the clearGTA dataset:

- **Act**: This paradigm is identical to ReAct but without the *thought* component.
- **ReAct** 3.5.1

For the baseline memory system, we used a linear memory implemented as a list. The first element of the list contains the system prompt, which instructs the SLM to solve a task using the provided tools and specifies the rules for tool usage. At each step, the SLM generates a thought followed by an action (for ReAct) or only an action (for Act). We append each step to the memory, including the thought (for ReAct), the tool request action, and the environment observation. The updated memory is then provided as the prompt for the next step. This iterative process continues until either a predefined `max_steps` is reached or the environment responds with success.

Due to time constraints and limited computational resources, we were unable to run MiniMindAgent method on the entire ClearGTA dataset. However, we tested it on multiple samples and obtained interesting preliminary insights. We found out that while the method presents some limits, the results are promising for future improvements.

We used a relatively small SLM with 1.5B parameters, aiming to select the smallest model that can still perform reasonably well on simple tasks. It is important to emphasize that such models are one to two orders of magnitude smaller than those typically employed in industry or in benchmarks like GTA. Therefore, we expect the model to perform poorly on such tasks. Specifically, we chose `Qwen2.5-1.5B-Instruct` due to its extended context

length capability and its post-training on instruction-following datasets, which make it more suitable for tool-use experiments.

Another limitation, due to resource constraints, is that each agent was restricted to a maximum of six tool requests per task. However, this does not bias our conclusions, as the study's objective is to compare baseline agents rather than optimize absolute performance. In fact, only 3 out of 228 tasks require more than six tool requests to be solved, while the overwhelming majority (199 tasks) require fewer than four steps Wang et al. (2024a).

7.1.1 Comparison of Performance

We observed 23 successes out of 229 trials for ReAct, while Act achieved 19 successes out of 229 trials.

Table 3 summarizes the aggregate statistics for both paradigms.

Table 3: Aggregate statistics for ReAct and Act on the ClearGTA dataset.

Metric	**ReAct**	**Act**
Mean total duration	44.58	31.44
Median total duration	41.63	29.12
Mean step duration	7.87	5.45
Mean total tokens	11037.29	10417.17
Success rate	0.100	0.083

Analysis: From Table 3, we observe that ReAct slightly outperforms Act in terms of success rate (0.100 vs 0.083), confirming the conclusions of Yao et al. (2023b). ReAct also exhibits higher mean and median total durations, as well as longer mean step durations, indicating that generating thoughts increases computational effort per step. This is confirmed by the fact that ReAct generates more tokens on average (11037.29 vs 10417.17) in total per task.

Overall, these results indicate that adding thought reasoning improves performance even for smaller models than those used in Yao et al. (2023b).

7.1.2 Limits of linear forward thinking

We also observed that linear and forward thinking alone is insufficient for SLMs to reach a solution. Models tend to get stuck in initial steps without knowing how to combine intermediate results with other information to progress.

For both Act and ReAct, the success rate eventually drops to zero as the number of tokens and the overall duration increase (Figure 5). This indicates that linear reasoning struggles with problems requiring multiple sequential steps.

Figure 6 further supports this conclusion: as the number of steps grows, the model becomes increasingly prone to collapse or hallucination, often inventing non-existent tools. This behavior may stem from the challenges of longer context windows as well as the model's difficulty in determining the next action due to the complexity of the solution path.

The initial steps, however, are easier for the model to handle. At the beginning, the rate of valid tool requests—i.e., requests belonging to the "tool_result" class that successfully yield results from the environment—is significantly higher than in later steps. By the second step, the model already shows a tendency to prematurely output an incorrect final answer without sufficient reasoning. This limitation becomes particularly problematic in realistic environments where no external feedback is available to verify the correctness of answers, and the model has no opportunity for backtracking once an incorrect solution is given.

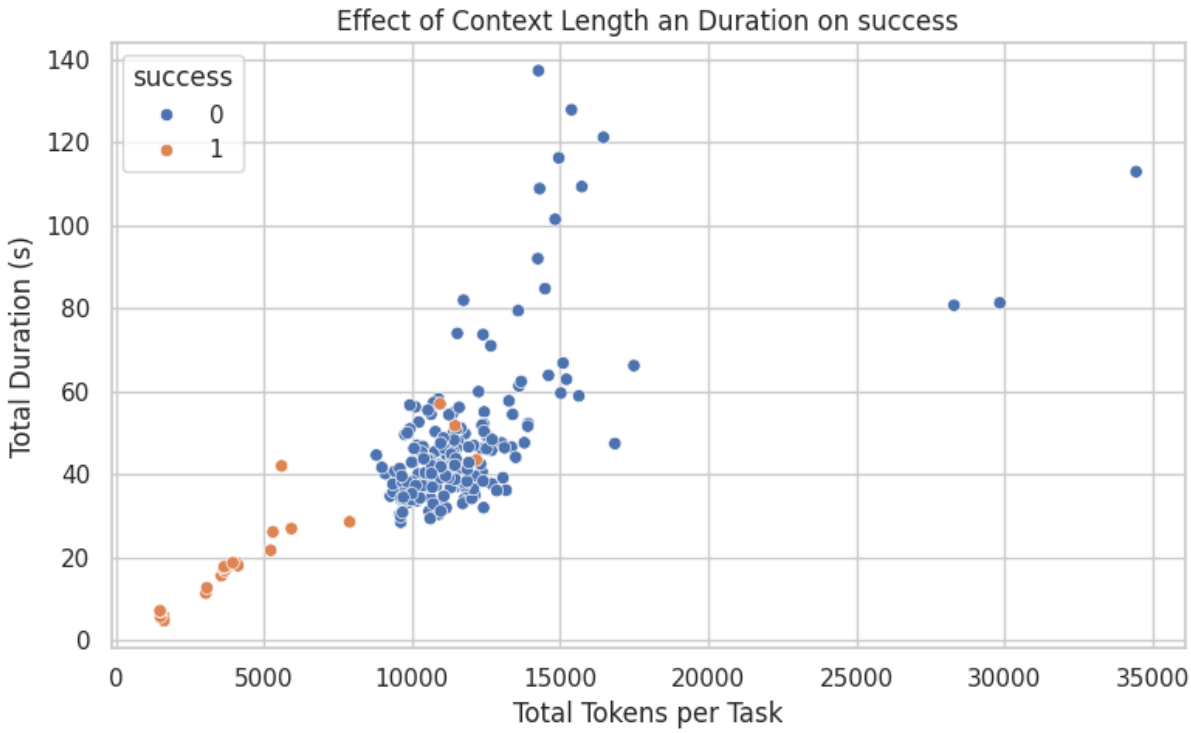


Figure 5: Influence of Total Context Length and Total Duration per Task on ReAct Agent Success.

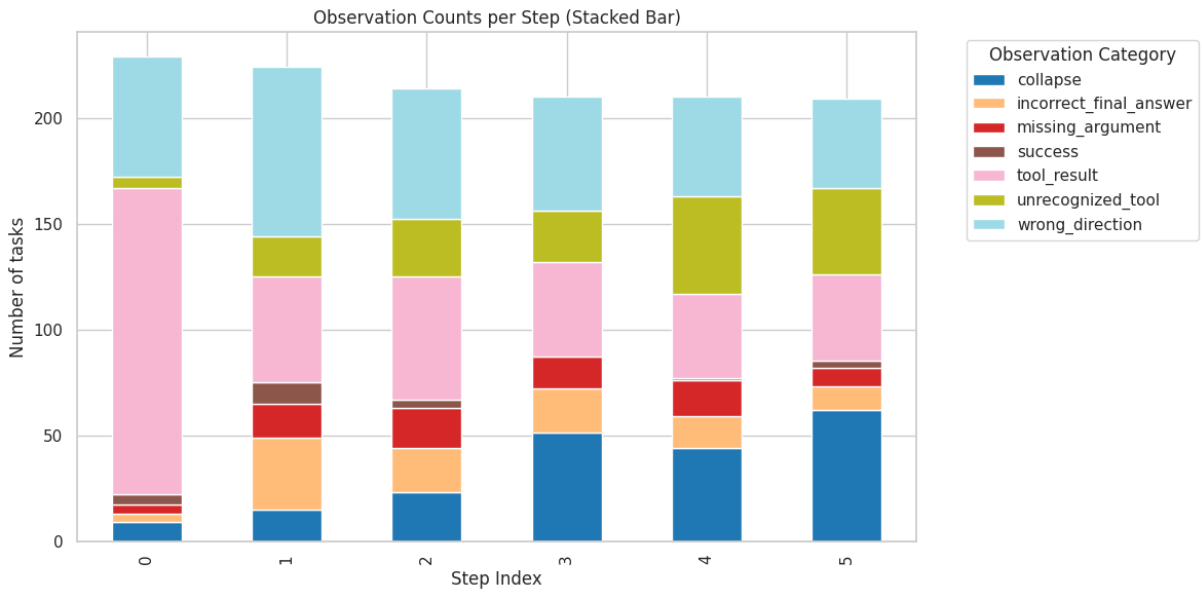


Figure 6: Stacked Bar Chart of encountered observation category for ReAct paradigm depending on the step index

7.2 MiniMindAgent

7.2.1 Limits of the New Methods

We identified several limitations when testing our new paradigm on samples from the ClearGTA dataset:

- Relying on multiple SLM calls to construct a reasoning tree increases the risk of entering loops of faulty reasoning. A single incorrect decision by one SLM call can propagate downstream. For example, if a child node is incorrectly judged as insufficient to answer the task, the system may continue to expand additional child nodes in search of alternative inputs, often leading to endless node creation until the depth limit is reached.
- Another risk is the distortion of information by child tools, which significantly affects the behavior of the parent. Since the parent node does not have access to the internal reasoning process of its children, crucial information may be distorted, removed, or forgotten.

7.2.2 Interesting Insights

Despite these limitations, the reasoning process of the new method can be considerably better than that of ReAct. By combining backward reasoning with forward reasoning,

the system maintains awareness of what is required and forces itself to identify what is missing from exploratory outputs in order to solve subtasks needed later.This self-reflective reasoning process is absent in the linear paradigm, which struggles to make effective use of intermediate discoveries. This is illustrated in Figures 7 and 8, generated with the networkx library by plotting, at each step, the memory tree of the MiniMindAgent for the task: "I want to buy a PS5 for each child in the photo. How many dollars will I need to spend in total?". Although the final answer for this task was incorrect due to the propagation of information distortion during SLM calls, the underlying reasoning and planning were correct.The MiniMindAgent's reasoning paradigm enables it to recognize from the beginning that solving the task requires both the number of children and the price of a PS5, each of which must be extracted using different tools, and then to combine these pieces of information to compute the total cost.

By contrast, the ReAct agent begins by hallucinating missing information, and when it receives feedback indicating that its answers are incorrect, it fails to recognize that the errors stem from missing information—specifically, the PS5 price. Rather than identifying and retrieving this missing value, ReAct repeatedly hallucinates new values for both the number of children and the PS5 price at each step, instead of engaging in deeper reasoning.

7.2.3 Promising Future Directions

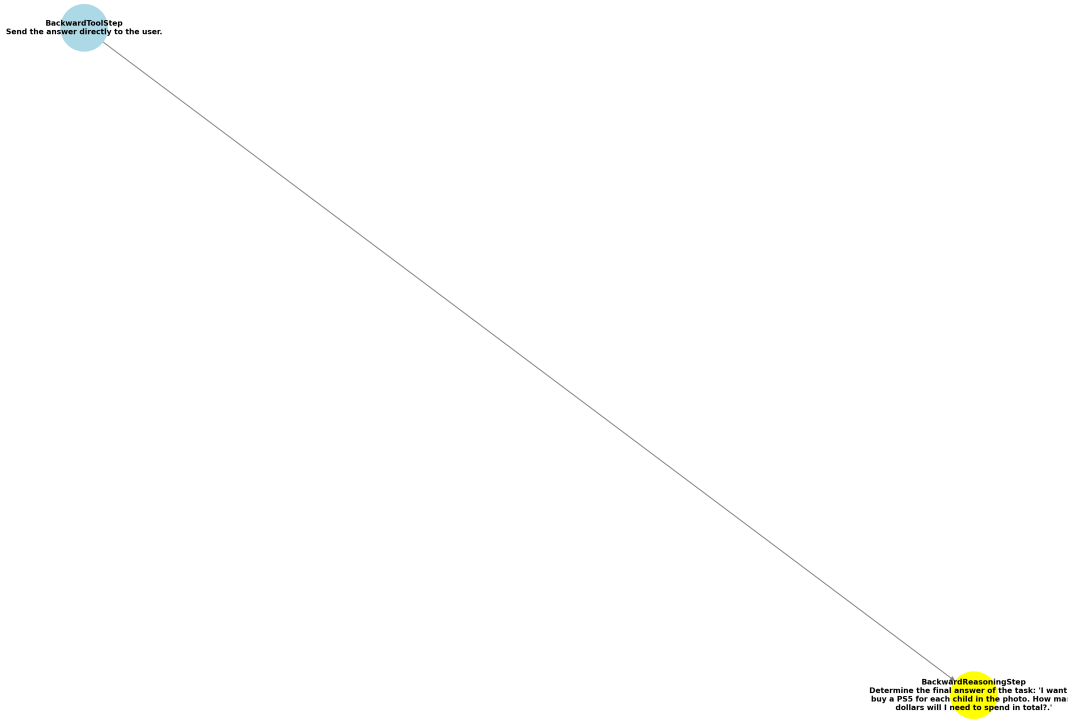
Although time constraints prevented us from implementing further improvements to the MiniMindAgent, several natural extensions emerged from our experiments. These include techniques to backtrack and identify the specific SLM call that introduced an error in reasoning:

Warnings of Failures The failure of a tool request can serve as an early signal of problematic reasoning. By maintaining the full chain of reasoning at each head, it becomes feasible to detect loops within the reasoning process. While detecting subtle reasoning errors is more challenging, it remains possible. An auxiliary SLM could be deployed at each step to monitor reasoning and generate alerts when issues are detected.

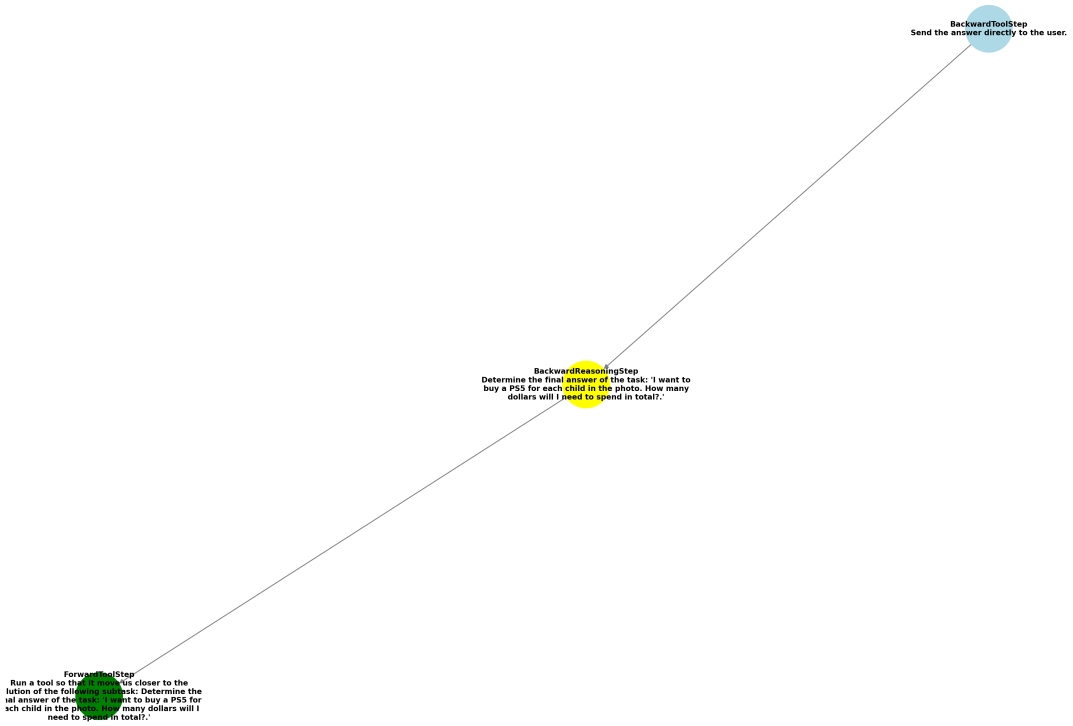
Tracing the Source of Errors Once an error in the reasoning chain has been localized, focus can shift to the corresponding node and its children. Additional SLM prompts can then be used to verify the validity of answers. If contradictions arise between the

verification step and the original response, a third SLM can be employed to adjudicate between them using explicit reasoning.

A New Training Paradigm This approach also suggests a novel training methodology for SLMs that does not rely on human annotation but instead leverages a series of tasks. Policy optimization techniques such as DPO or GRPO could be used to train SLMs to improve performance at each individual call, thereby strengthening the overall reasoning process.

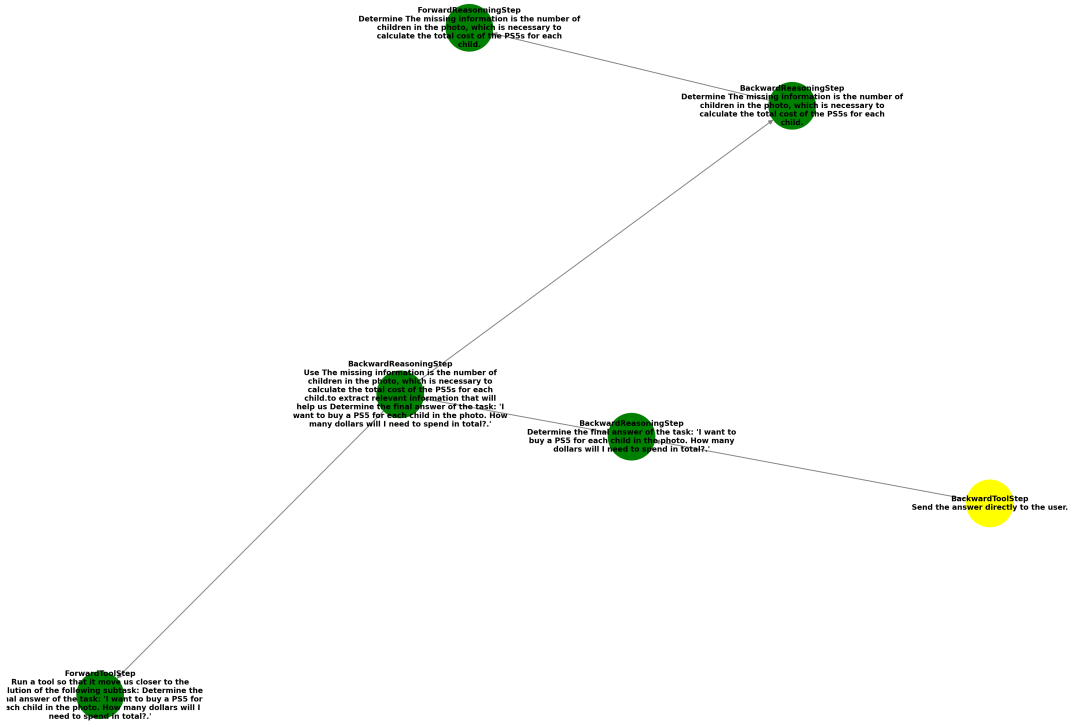


(a) Shortly after the first step



(b) Few steps further

Figure 7: Illustration of MiniMindAgent tree construction: First Steps. The color green indicates that the tool has been executed successfully. The color yellow corresponds to the head of the tree.



(a) Last Step

Figure 8: Illustration of MiniMindAgent tree construction: Last Step. The color green indicates that the tool has been executed successfully. The color yellow corresponds to the head of the tree.

8 CONCLUSION

8.0.1 Conclusion

In summary, our experiments with MiniMindAgent demonstrate both the potential and the current shortcomings of this new paradigm. While the reliance on multiple SLM calls introduces risks such as error propagation, information distortion, and looping behaviors, the approach also enables richer reasoning than linear methods like ReAct. By integrating forward and backward reasoning, MiniMindAgent is able to maintain focus on what is required for solving subtasks, thereby increasing the tendency to push the reasoning further.

The preliminary insights suggest that additional mechanisms—such as error detection, backtracking, and verification through auxiliary SLMs—could significantly enhance robustness. Furthermore, the paradigm opens new possibilities for training strategies that

rely on task-based feedback rather than human annotation, using reinforcement-style optimization techniques such as PPO, DPO or GRPO. Overall, although still at an early stage, MiniMindAgent shows promise as a foundation for more resilient and adaptive reasoning systems in the future.

This work also opens several promising directions for future research:

- Continue improving the MiniMindAgent by enhancing it with backtracking tools and testing it on the full ClearGTA dataset.
- Experiment with the new training paradigm on MiniMindAgent and compare its performance against baseline training paradigms or ReAct.
- Compare the performance of these paradigms with standard agent paradigms such as PlanAct and ReWOO, or under alternative memory systems such as long-term memory or scratchpads.
- Explore additional dimensions of AI agent challenges as outlined in the taxonomy of problems.

References

Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., and Sanghai, S. (2023). Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*.

Ben Allal, L., Lozhkov, A., Bakouch, E., Blázquez, G. M., Penedo, G., Tunstall, L., Marafioti, A., Kydlíček, H., Piqueres Lajarín, A., Srivastav, V., et al. (2025). Smollm2: When smol goes big – data-centric training of a small language model. *arXiv preprint arXiv:2502.02737*.

Clark, P., Gardiner, E., Coecke, B., and et al. (2019). Boolq: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 2924–2936. ACL.

Clark, P., Tafjord, O., and Richardson, M. (2018). Think you have solved question answering? try arc, the ai2 reasoning challenge. In *AAAI Conference on Artificial Intelligence*.

Guo, D., Zhou, C., Hu, Z., and Zhang, R. (2023). Lambada: Backward-chaining language model reasoning. In *ACL*.

Hao, S., Gu, Y., Ma, H., Hong, J. J., Wang, Z., Wang, D. Z., and Hu, Z. (2023). Reasoning with language model is planning with world model. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 8154–8173. Association for Computational Linguistics.

Hendrycks, D., Burns, C., Basart, S., and et al. (2021). Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*.

Hoffmann, J., Borgeaud, S., Mensch, A., et al. (2022). Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. (2020). Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.

Kim, J., Park, H., and Lee, M. (2024). Small language models learn enhanced reasoning skills from medical textbooks. *Journal of Artificial Intelligence Research*, 73:1–28.

Li, F., Sun, Q., and Wang, R. (2023a). Starcoder: Open large language model for code generation. arXiv:2305.12345.

Li, T., Zhou, Y., Su, H., and Chen, W. (2023b). Plan-and-solve prompting for zero-shot reasoning. *arXiv preprint arXiv:2310.00405*.

Li, Y., Liu, X., Wu, C., and Yu, D. (2023c). Rewoo: Reasoning without observation for efficient multi-step reasoning. *arXiv preprint arXiv:2306.00021*.

Lu, Z., Li, X., Cai, D., Yi, R., Liu, F., Zhang, X., Lane, N. D., and Xu, M. (2024). Small language models: Survey, measurements, and insights. *arXiv preprint arXiv:2409.15790*.

Mihaylov, T. and Clark, P. (2018). Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2381–2391. ACL.

OpenBMB (2024). Toolbench: An open platform for training, serving, and evaluating large language models for tool learning. Accessed: 2025-09-01.

Rafailov, R., Kumar, A., McKinzie, B., Risteski, A., Finn, C., and Ermon, S. (2023). Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*. Accessed: 2025-09-01.

Shao, Z., Liu, X., Xu, C., Wang, Z., Sun, Q., Zhang, Y., and Chen, K. (2024). Group relative policy optimization. *arXiv preprint arXiv:2402.00856*. Accessed: 2025-09-01.

Shazeer, N. (2020). Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*. Accessed: 2025-09-01.

Su, J. (2023). Qkv bias in attention mechanisms. *arXiv preprint arXiv:2309.15723*. Accessed: 2025-09-01.

Su, J., Lu, Y., Pan, S., Wen, B., and Liu, Y. (2021). Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*. Accessed: 2025-09-01.

Talmor, A., Herzig, J., Lourie, N., and Berant, J. (2019). Commonsenseqa: A question answering challenge targeting commonsense knowledge. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, pages 4149–4158. ACL.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., and Lample, G. (2023). Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*. Accessed: 2025-09-08.

Wang, J., Ma, Z., Li, Y., Zhang, S., Chen, C., Chen, K., and Le, X. (2024a). Gta: A benchmark for general tool agents. *arXiv preprint arXiv:2407.08713*. Accessed: 2025-09-01.

Wang, P., Wu, Y., Wang, Z., Liu, J., Song, X., Peng, Z., Deng, K., Zhang, C., Wang, J., Peng, J., Zhang, G., Guo, H., Zhang, Z., Su, W., and Zheng, B. (2024b). Mtu-bench: A multi-granularity tool-use benchmark for large language models. *arXiv preprint arXiv:2410.11710*. Accessed: 2025-09-01.

Wang, Z., Cui, Y., Zhong, L., Zhang, Z., Yin, D., Lin, B. Y., and Shang, J. (2024c). Officebench: Benchmarking language agents across multiple applications for office automation. *arXiv preprint arXiv:2407.19056*. Accessed: 2025-09-01.

Wenzek, G., Nguyen, T., Joulin, A., and Grave, E. (2020). Ccnet: Extracting high quality monolingual datasets from web crawl data. arXiv:2004.09432.

Wu, S., Bao, H., Kunievsky, N., and Evans, J. A. (2025). Introspective growth: Automatically advancing llm expertise in technology judgment. *arXiv preprint arXiv:2505.12452*.

Xu, T., Zhang, L., and Chen, Y. (2023). Fineweb-edu: Large language models for educational dataset curation. *arXiv preprint arXiv:2303.01234*.

Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., Lin, H., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Lu, K., Bao, K., Yang, K., Yu, L., Li, M., Xue, M., Zhang, P., Zhu, Q., Men, R., Lin, R., Li, T., Tang, T., Xia, T., Ren, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Wan, Y.,

Liu, Y., Cui, Z., Zhang, Z., and Qiu, Z. (2024). Qwen2.5 technical report. Technical report, arXiv preprint arXiv:2412.15115.

Yao, X., Jiang, S., Zhao, Y., Dai, H., and Zhou, M. (2023a). Tree of thoughts: Deliberate problem solving with large language models. In *NeurIPS*.

Yao, X., Peng, N., Roy, S., Wang, R., and Liang, P. (2023b). React: Synergizing reasoning and acting in language models. In *Proceedings of NeurIPS*.

Zellers, R., Bisk, Y., Farhadi, A., and Choi, Y. (2019). Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800. ACL.

Zhang, B. and Sennrich, R. (2019). Root mean square layer normalization. *arXiv preprint arXiv:1910.07467*. Accessed: 2025-09-01.

Zhong, L., Du, Z., Zhang, X., Hu, H., and Tang, J. (2025). Complexfuncbench: Exploring multi-step and constrained function calling under long-context scenario. Accessed: 2025-09-01.